

# RAG-Based Postmortem Intelligence System

## Individual Contributions Report

RAG\_Major\_Project — July 20, 2026 to August 29, 2026

This report documents the assigned roles and individual contributions of the currently known team members for the RAG-Based Postmortem Intelligence System, based on the project report

*RAG\_Postmortem\_Intelligence\_Weekly\_Daily\_Report\_July20\_Aug29\_2026.md*. Contribution links point to the new repository [https://github.com/VIDUSHH/RAG\\_Major\\_Project.git](https://github.com/VIDUSHH/RAG_Major_Project.git).

A fourth team member will be documented in this repository later once their information is provided. No fourth member is included in this report at this time.

## 1. Vidush Dabeer

GitHub: <https://github.com/VIDUSHH> | Email: vidushdabeer06@gmail.com

### Assigned Role

**RAG / Backend Lead** — Primary engineer for the core RAG pipeline and all backend services (RAG architecture, FastAPI, Qdrant, Neo4j, LLM grounding, embeddings, hybrid retrieval).

### Contribution Summary

Designed the dual-plane (FastAPI/Django) system architecture and the RAG pipeline flow: query understanding → hybrid retrieval → graph retrieval → context assembly → LLM generation → grounding.

- Implemented the embedding provider abstraction (local sentence-transformers + Gemini), the hybrid retriever (dense Qdrant + sparse BM25 + Reciprocal Rank Fusion), the graph retriever, and the query understanding engine.
- Implemented the LLM provider with server-side grounding (prevents fabricated citations), the RAGPipeline orchestrator, and all query/analysis endpoints.
- Built the Qdrant vector service (collection management, payload indexes, dimension-mismatch recreation) and the core Neo4j graph service (nodes, relationships, schema/constraints).
- Owned the ingestion pipeline, multi-tenancy enforcement, and the event-loop freeze fix; wrote the security, LLM grounding, ingestion, and activity test suites.

### Relevant Contributed Files

- fastapi\_backend/app/rag/pipeline.py, rag/models.py
- fastapi\_backend/app/retrieval/hybrid.py, retrieval/graph\_retrieval.py
- fastapi\_backend/app/embeddings/provider.py
- fastapi\_backend/app/llm/provider.py
- fastapi\_backend/app/services/vector\_service.py, services/graph\_service.py, services/activity\_service.py
- fastapi\_backend/app/api/v1/query.py, api/v1/ingestion.py
- fastapi\_backend/app/core/config.py, core/security.py, core/dependencies.py
- django\_backend/apps/core/clients.py, apps/accounts, apps/organizations, apps/projects
- tests/fastapi\_backend/test\_security.py, test\_llm.py, test\_ingestion.py, test\_activity\_service.py, test\_health.py

### Lessons Learnt

- Designing the embedding provider as an abstraction enables hot-swapping between Gemini API and local sentence-transformers, saving quota and enabling offline operation.
- Synchronous blocking calls inside async def handlers freeze the FastAPI event loop; converting handlers to plain def runs blocking work safely in worker threads.
- Building the BM25 corpus lazily from Qdrant scroll (with periodic refresh) keeps ingestion write-only while delivering true hybrid retrieval.
- Server-side grounding of LLM output (rebuilding evidence-derived fields from actual chunks) structurally prevents fabricated citations.

## Skills Learnt

RAG architecture and pipeline design; FastAPI and threaded execution; Qdrant vector store; Neo4j graph database; hybrid retrieval (BM25 + dense + RRF); LLM integration and grounding (Gemini + mock); sentence-transformers embeddings; pytest; multi-tenancy enforcement.

## Contribution / File Links

- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/rag](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/rag)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/retrieval](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/retrieval)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/llm](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/llm)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/embeddings](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/embeddings)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/services](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/services)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/fastapi\\_backend/app/api/v1](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/fastapi_backend/app/api/v1)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/tests/fastapi\\_backend](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/tests/fastapi_backend)

## 2. Samarth Joshi

GitHub: <https://github.com/samarthjoshi812> | Email: samarthjoshi812@gmail.com

## Assigned Role

**Configuration / DevOps Support** — Supporting role for configuration, API key management, networking, and documentation (API/Configuration and Documentation).

## Contribution Summary

Provided supporting contributions across configuration, API key management, testing, and documentation.

- Obtained and managed the Gemini API key and the internal API key; created and maintained .env / .env.example.
- Verified Docker Compose networking (FastAPI & Qdrant/Neo4j by container name) and assisted with Docker troubleshooting.
- Helped test parsers, chunking metadata, BM25 keyword queries, and query-intent detection; reported visual and functional SPA issues.
- Documented test results, known limitations, and updated configuration references throughout the project.

## Relevant Contributed Files

- .env.example (maintained environment configuration template)
- infrastructure/docker-compose.yml (verified Docker networking/topology)

- README.md, docs/architecture.md, docs/ingestion\_guide.md (documentation/config references)
- fastapi\_backend/app/core/config.py (configuration references verified)
- Test-suite verification across tests/ (parsers, chunking metadata, BM25, query intent)

## Lessons Learnt

- Centralizing all environment configuration in .env/.env.example with a single Settings object (pydantic-settings) keeps the project consistent and easy to set up.
- Docker services communicate by container name on a shared compose network; verifying FastAPI & Qdrant/Neo4j connectivity isolates networking misconfigurations early.
- Documenting test results, API-key usage, and known limitations as the project evolves keeps configuration references accurate for the whole team.

## Skills Learnt

Environment configuration management (.env/.env.example, pydantic-settings); API key management (Gemini and internal service-to-service keys); Docker Compose networking and troubleshooting; testing parsers, chunking metadata, BM25 keyword queries, and query-intent detection; technical documentation.

## Contribution / File Links

- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/.env.example](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/.env.example)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/infrastructure/docker-compose.yml](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/infrastructure/docker-compose.yml)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/README.md](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/README.md)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/docs/architecture.md](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/docs/architecture.md)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/docs/ingestion\\_guide.md](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/docs/ingestion_guide.md)

## 3. Abdul Rehman

GitHub: <https://github.com/abdulrehmansoudagar01-dev>

### Assigned Role

**Testing / Validation Support** — Supporting role for testing, edge-case validation, and data verification (Testing/QA, parsers & retrieval validation, Neo4j verification, integration checks).

### Contribution Summary

Provided supporting contributions across configuration, testing, and validation.

- Tested parser output against dataset files and verified chunk metadata accuracy across log formats.
- Tested hybrid retrieval edge cases (empty corpus, single-document corpus, long queries) and Neo4j dependency extraction.
- Tested API-key/auth edge cases (missing/wrong/bearer), empty-collection and nonexistent-document cases, and large-file/quota limits.
- Ran the full test suite (127 tests) repeatedly, verified no regressions, and validated Neo4j constraints/indexes and cache-busting behavior.

### Relevant Contributed Files

- tests/ (full test suite run and validated), tests/conftest.py
- fastapi\_backend/app/ingestion/parsers.py (parser output and edge cases validated)

- `fastapi_backend/app/retrieval/hybrid.py` (retrieval edge cases tested)
- `fastapi_backend/app/retrieval/graph_retrieval.py`, `services/graph_service.py` (Neo4j verified)
- `fastapi_backend/app/core/security.py` (API-key/auth edge cases tested)
- `fastapi_backend/app/static/app.js` (cache-busting behavior verified)

## Lessons Learnt

- Rigorous edge-case testing of parsers and retrieval (empty corpus, single-document corpus, long queries, missing/wrong API keys, empty collections) surfaces bugs that happy-path testing misses.
- Validating the full 127-test suite repeatedly and checking for regressions after each change keeps the project stable and demo-ready.
- Verifying Neo4j constraints/indexes and cache-busting behavior confirms infrastructure changes behave as expected end to end.

## Skills Learnt

Testing/QA and edge-case validation; log parser output verification across formats; hybrid retrieval validation (BM25 + dense + RRF edge cases); Neo4j dependency extraction and constraint/index verification; API-key and authentication edge-case testing; test-suite execution and regression checking (127 tests); cache-busting / UI behavior verification.

## Contribution / File Links

- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/tree/main/tests](https://github.com/VIDUSHH/RAG_Major_Project/tree/main/tests)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/tests/conftest.py](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/tests/conftest.py)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/fastapi\\_backend/app/ingestion/parsers.py](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/fastapi_backend/app/ingestion/parsers.py)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/fastapi\\_backend/app/retrieval/hybrid.py](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/fastapi_backend/app/retrieval/hybrid.py)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/fastapi\\_backend/app/retrieval/graph\\_retrieval.py](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/fastapi_backend/app/retrieval/graph_retrieval.py)
- [https://github.com/VIDUSHH/RAG\\_Major\\_Project/blob/main/fastapi\\_backend/app/services/graph\\_service.py](https://github.com/VIDUSHH/RAG_Major_Project/blob/main/fastapi_backend/app/services/graph_service.py)